

[Home](#) > [Functional Programming, SWU \(Spring 2026\)](#) > [Assignment 2](#) [Assignment 2](#) Available since 2 months · Due a month ago · Locked a month ago · [Grades published](#)

Assignment description

[Assignment](#)[Home](#)[Assignment description](#)[Rubric](#)[Course feedback](#)

Assignment 2

Solve the following exercises. Any non-code part of the assignment must be handed in as comments in the source code.

You write multiline comments between `(*` and `*)` and single-line comments using `//`.

Some of the exercises (which are marked) are copied more or less verbatim from the course book, for your convenience, possibly with minor reformulations.

Use the template [Assignment2.fs](#) and upload that file. There are a few important things to note.

- The module title must be `Assignment2` or the project will not compile
- All functions have a placeholder with a `failwith "not completed"`. This placeholder must be kept for all of the assignments that you have not solved or the project will not compile.
- None of the list functions may make use of higher-order functions (like `fold`, `filter`, or `map`). We will cover these next week.

Green exercises

Exercise 2.1

Write a function `downto1 : int -> int list` that given an integer n returns the n -element list `[n; n-1; ...; 1]` if $n > 0$ and `[]` otherwise. You must use if-then-else expressions to define the function.

Secondly define the function `downto2` which behaves in exactly the same way as `downto1`. This time you must use pattern matching using `match`.

Finally, define the function `downto3` which behaves in exactly the same way as `downto1` and `downto2`. This time you must use pattern matching using `function`.

Examples

```
> downto1 5;;
- val it : int list = [5; 4; 3; 2; 1]

> downto2 7;;
- val it : int list = [7; 6; 5; 4; 3; 2; 1]

> downto3 10;;
- val it : int list = [10; 9; 8; 7; 6; 5; 4; 3; 2; 1]

> downto1 0;;
- val it : int list = []

> downto2 -42;;
- val it : int list = []
```

Exercise 2.2

Write a function `removeOddIdx : 'a list -> 'a list` that given a list `xs` returns a list where all

odd-indexed elements of **xs** have been removed.

```
// type annotation most likely required here
> removeOddIdx ([] :: Int list);;
- val it :: Int list = []

> removeOddIdx [true];;
- val it :: bool list = [true]

> removeOddIdx ["Marry"; "had"; "a";
"little"; "lamb"; "its"; "fleece";
..... "was"; "white"; "as";
"snow"];;
- val it :: string list = ["Marry"; "a";
"lamb"; "fleece"; "white"; "snow"]
```

Exercise 2.3

Write a function **combinePair** : 'a list -> ('a * 'a) list that given a list **xs** returns the list with elements from **xs** combined into pairs. If **xs** contains an odd number of elements, then the last element is thrown away.

Hint: Use pattern matching

Examples:

```
// type annotation most likely required here
> combinePair ([] :: Int list);;
- val it :: (int * int) list = []

> combinePair [true; false];;
- val it :: (bool * bool) list = [(true, false)]
```

```
> combinePair ["Marry"; "had"; "a";
"little"; "lamb"; "its"; "fleece";
..... "was"; "white"; "as";
"snow"];;
- val it :: (string * string) list =
  .. [("Marry", "had"); ("a", "little");
("lamb", "its");
.... ("fleece", "was"); ("white", "as")]
```

Exercise 2.4 (based on exercise 3.3 from HR)

The set of *complex numbers* is the set of pairs of real numbers.

Define a type **complex** that represents complex numbers with floating point components.

Define a function **mkComplex : float -> float -> complex** that given two floating point numbers return the corresponding complex number.

Define a function **complexToPair : complex -> float * float** that given a complex number (a, b) returns the pair (a, b) .

When doing the following exercises make sure that you use let-statements to not have to repeat commonly occurring calculations.

Addition and multiplication

addition and multiplication are defined by:

$$\begin{aligned}(a, b) + (c, d) &= (a + c, b + d) \\ (a, b) * (c, d) &= (ac - bd, bc + ad)\end{aligned}$$

Declare infix functions **|+|** and **|*|** of type **complex -> complex -> complex** for addition

and multiplication of complex numbers.

Division and subtraction

The additive and multiplicative inverse of a complex number c , that is $-c$ and $1/c$ respectively, are defined by:

$$-(a, b) = (-a, -b)$$

$$\frac{1}{(a, b)} = \begin{cases} \left(\frac{a}{a^2+b^2}, \frac{-b}{a^2+b^2} \right) & \text{if } a \neq 0 \text{ or } b \neq 0 \\ \text{undefined} & \text{otherwise} \end{cases}$$

Declare infix functions `| - |` and `| / |` of type `complex -> complex -> complex` for subtraction and division of complex numbers.

Hint: Use addition and multiplication with their inverses ($a - b = a + (-b)$ for instance).

Hint: Do not worry about the **undefined** cases at all. Your program will most likely crash if you try to get the multiplicative inverse of $(0, 0)$, since you will divide by zero, which is completely standard. You may, however, add your own exception to handle this case if you feel that makes your code cleaner.

Exercise 2.5

Write a non-recursive function `explode1 : string -> char list` that given a string `s` returns the list of characters in `s`.

Examples:

```
> explode1 "";;
- val it : char list = []
```

```
> explode1 "Hello World!";;
- val it : char list =
  ..... ['H'; 'e'; 'l'; 'l'; 'o'; ' '; 'W';
  'o'; 'r'; 'l'; 'd'; '!']
  ....
```

Hint: if `s` is a string then `s.ToCharArray()` returns an array of characters. You can then use `List.ofArray` to turn it into a list of characters.

Now write a recursive function `explode2` : `string -> char list` that behaves the same as `explode` except that you now have to use the string function `s.Chars` (or `. [index]`), where `s` is a string. You can also make use of `s.Remove(0,1)`.

Exercise 2.6

Write a function `implode` : `char list -> string` that given a list of characters `cs` returns a string with all characters of `cs` in the same order.

Write a function `implodeRev` : `char list -> string` that given a list of characters `cs` returns a string with all characterl of `cs` in reverse order. You may not use `List.rev` for this exercise.

Examples:

```
> implode [];;
- val it : string = ""

> implode ['H'; 'e'; 'l'; 'l'; 'o'; ' '; 'W';
  'o'; 'r'; 'l'; 'd'; '!'];;
- val it : string = "Hello World!"

> implodeRev [];;
```

```
- val it : string = ""

> implodeRev ['H'; 'e'; 'l'; 'l'; 'o'; ' ';
'; 'W'; 'o'; 'r'; 'l'; 'd'; '!'];
- val it : string = "!dlroW olleH"
```

Exercise 2.7

Write a function `toUpper : string -> string` that given a string `s` returns `s` with all characters in upper case.

Hint: Use `System.Char.ToUpper` to convert characters to upper case. You can do this function in one line using `implode`, `explode`, and a function of your own that iterates over a list to turn all characters into upper case.

This is a good exercise to play around with piping `|>` and function composition `>>`. With piping, for instance, you can start your function with

```
let toUpper s = s |> ...
```

If you use function composition you can even get away without naming your parameter at all:

```
let toUpper = ... >> ...
```

Feel free to hand in several solutions to this exercise if you want feedback. It is useful to get familiar with these connectives.

Exercise 2.8

The Ackermann function is a recursive function where both value and number of mutually recursive calls grow rapidly.

Write the function `ack : int * int -> int` that given an integer pair `(m, n)` implements the Ackermann function using pattern matching on the cases of $A(m, n)$ as given below.

$$A(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

Notice that the Ackermann function is defined for non-negative numbers only.

Make sure that your pattern matching is exhaustive.

Yellow exercises

Exercise 2.9

Create a function `reverse` that given a list `lst` returns `lst` reversed. Your function must not use any auxiliary, internal, or standard library functions.

Hint: You can append singleton elements to the end of a list by using constructs `xs @ [x]`. This is not an efficient way, since this operation in this case is linear with respect to the length of `xs` but we will cover how to do this efficiently in later exercises.

Examples:

```
> reverse ([] :: int list) (* The type
annotation is necessary here or F# will
complain that it does not know what type
of elements are possible to add to the
empty list. *)
- val it: int list = []
```



```
> reverse [5]
- val it: int list = [5]

> reverse [1; 4; 2; 23]
val it: int list = [23; 2; 4; 1]
```

Exercise 2.10

Create a function `palindrome` of type `string -> bool` that given a string `str` returns `true` if `str` is the same forwards as backwards.

Hint: Use `reverse` and `explode1` or `explode2`

Examples:

```
> palindrome "sirap i paris"
- val it: bool = true

> palindrome "sirap i Paris"
- val it: bool = false

> palindrome "a!bcb!a"
- val it: bool = true
```

Exercise 2.11

Create a function `keepLetters` of type `char list -> char list` that given a list of characters `lst` removes all characters in `lst` that are not a letter in the alphabet according to the standard library function `System.Char.IsAsciiLetter`. You may not use any auxiliary, internal, or standard library functions (other than `System.Char.IsAsciiLetter`).

Examples:

```
> keepLetters []
- val it: char list = []

> keepLetters ['a'; 'b'; '!'; 'c'; 'æ']
- val it: char list = ['a'; 'b'; 'c']
```

Exercise 2.12

Create a function `palindrome2` that works exactly the same as `palindrome` but that treats lower- and upper case characters the same and ignores non-letters, according to the `System.Char.IsAsciiLetter` function from the standard library.

Hint: use `toUpper` from the green exercises.

Examples:

```
> palindrome2 "sirap i paris"
- val it: bool = true

> palindrome2 "sirap i Paris"
- val it: bool = true

> palindrome2 "Dromedaren Alpotto planerade mord!!!"
- val it: bool = true

> palindrome2 "Dromedaren Alpotto er sød."
val it: bool = false
```

Red Exercises

Exercise 2.13

Create a function `palindrome3` that works exactly

like `palindrome2` but that

- does **not** convert the string to a list to help with the computation.
- uses `str[i]` to access a character at index `i` from the string `str`
- terminates as soon as it realises that the string is not a palindrome, i.e. it will not make any more comparisons than are strictly necessary.

Hint: Work from both directions. Compare the starting with the ending character, if these are the same keep going with the next two characters, and if not terminate. You must also handle the case where one (or both) of the characters you are comparing is not a letter and should be ignored.

© 2026 - CodeGrade ([Revanche](#)) - Made with  - [Privacy statement](#) - [Help](#)